



Mata Kuliah: **Pengolahan Sinyal Digital (IF3024)**

Tugas : **Tugas Besar**

Nama Anggota:

Tanggal: **May 31, 2025**

1. **Bintang Fikri Fauzan (122140008)**
 2. **Elsa Elisa Yohana Sianturi (122140135)**
 3. **Eden Wijaya (122140187)**
-

Implementasi Metode *Plane-to-Orthogonal Skin* (POS) untuk Estimasi *Real-Time Heart Rate* dan *Respiration Rate* melalui Webcam

1 Pendahuluan

1.1 Latar Belakang

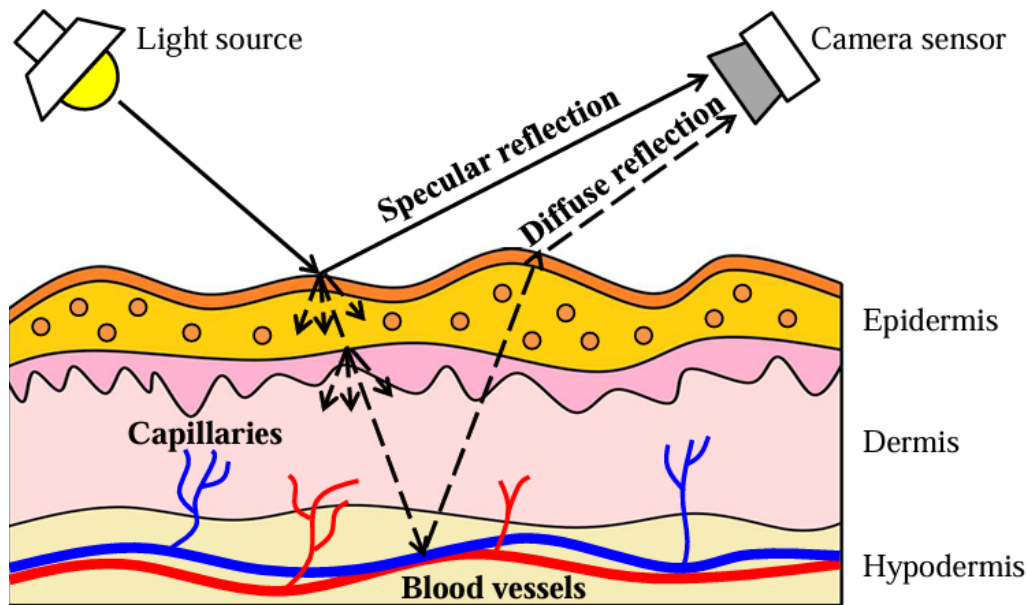
Pemantauan kondisi fisiologis seperti denyut jantung (*heart rate/HR*) dan laju pernapasan (*respiration rate/RR*) merupakan aspek penting dalam bidang kesehatan dan kebugaran. Metode konvensional seperti elektrokardiografi (ECG) dan *photoplethysmography* (PPG) umumnya memerlukan kontak langsung dengan kulit menggunakan sensor atau elektroda, yang dapat mengurangi kenyamanan pengguna terutama dalam pemantauan jangka panjang [1]. Sebagai alternatif, metode *remote photoplethysmography* (rPPG) menawarkan pendekatan non-kontak yang menggunakan kamera untuk merekam perubahan warna kulit akibat fluktuasi aliran darah. rPPG memungkinkan akuisisi sinyal denyut jantung dari video wajah yang direkam dengan webcam, tanpa perlu menyentuh kulit secara langsung [1]. Namun, akurasi metode ini sangat dipengaruhi oleh variasi pencahayaan, gerakan kepala, warna kulit, dan ekspresi wajah [1]. Untuk mengatasi permasalahan tersebut, salah satu pendekatan yang menjanjikan adalah metode *Plane-to-Orthogonal Skin* (POS), yang pertama kali diperkenalkan oleh Wang et al., dan dikembangkan lebih lanjut dalam berbagai penelitian. Metode ini memproyeksikan sinyal RGB ke bidang ortogonal terhadap vektor warna kulit, sehingga dapat mengurangi pengaruh pencahayaan dan meningkatkan keandalan ekstraksi sinyal denyut [2].

Dengan mempertimbangkan keunggulan metode POS dalam meningkatkan akurasi estimasi HR dan RR dari video wajah, serta potensinya dalam aplikasi monitoring kesehatan non-invasif dan real-time, maka tugas besar ini mengimplementasikan metode POS untuk mengestimasi denyut jantung dan laju pernapasan secara real-time melalui webcam.

1.2 Rumusan Masalah

Berdasarkan latar belakang yang telah dipaparkan, maka rumusan masalah dalam tugas besar ini adalah sebagai berikut:

1. Bagaimana cara mengimplementasikan metode *Plane-to-Orthogonal Skin* (POS) untuk estimasi denyut jantung dan laju pernapasan secara real-time menggunakan webcam?
2. Bagaimana gerakan naik-turun bahu pada video dapat digunakan untuk mendeteksi dan mengestimasi laju pernapasan (*respiration rate*)?



Gambar 1: Plane-to-Orthogonal Skin

3. Bagaimana perubahan warna kulit wajah dalam video dapat dimanfaatkan untuk mengestimasi denyut jantung (*heart rate*) secara non-kontak?

1.3 Tujuan

Adapun tujuan dari tugas besar ini adalah sebagai berikut:

1. Mengimplementasikan metode *Plane-to-Orthogonal Skin* (POS) untuk mengestimasi HR dan RR secara real-time melalui video dari webcam.
2. Menganalisis kemampuan sistem dalam mendeteksi laju pernapasan berdasarkan pola pergerakan vertikal (naik-turun) bahu yang terekam dalam video.
3. Mengevaluasi efektivitas perubahan warna kulit wajah dalam mendeteksi sinyal denyut jantung melalui metode pemrosesan citra non-kontak.

2 Alat dan Bahan

Proyek tugas besar ini dikembangkan sepenuhnya dalam lingkungan perangkat lunak dengan memanfaatkan bahasa pemrograman Python serta beberapa pustaka pendukung untuk pemrosesan video, sinyal, serta antarmuka pengguna (GUI). Berikut adalah alat dan bahan yang digunakan:

2.1 Bahasa Pemrograman dan Lingkungan Pengembangan

1. Python 3.10, digunakan sebagai bahasa pemrograman utama karena mendukung pustaka sains dan citra digital secara luas.
2. Visual Studio Code / Terminal, digunakan sebagai editor dan lingkungan pengembangan lokal.

2.2 Library dan Pustaka yang Digunakan

1. OpenCV-python 4.8.1.78, digunakan untuk pemrosesan video dan citra secara real-time dari webcam.
2. MediaPipe 0.10.7, digunakan untuk deteksi landmark wajah dan tubuh, termasuk gerakan bahu serta area kulit wajah.
3. PyQt5 5.15.9, digunakan untuk membangun antarmuka pengguna (GUI) dalam bentuk aplikasi desktop.
4. Numpy 1.24.3, digunakan untuk manipulasi array numerik dan perhitungan matematis dasar.
5. Scipy 1.11.3, digunakan untuk analisis sinyal digital seperti filter dan transformasi Fourier.
6. Matplotlib 3.7.2, digunakan untuk visualisasi grafik sinyal denyut jantung dan respirasi.

2.3 Metode dan Algoritma

1. Estimasi *Heart Rate* (HR), dilakukan dengan mengekstrak sinyal warna dari area dahi pada wajah dan menganalisis menggunakan transformasi Fourier untuk mendeteksi frekuensi denyut jantung.
2. Estimasi *Respiration Rate* (RR), dilakukan dengan melacak pergerakan vertikal (naik-turun) bahu dari frame video menggunakan landmark yang disediakan oleh MediaPipe.
3. Transformasi Fourier (FFT), digunakan untuk mengubah sinyal waktu menjadi domain frekuensi, sehingga denyut jantung dan pernapasan dapat dihitung dari spektrum dominan.

3 Penjelasan

Struktur folder proyek ini dirancang untuk mendukung pengembangan sistem estimasi *heart rate* dan *respiration rate* secara real-time menggunakan webcam. Direktori utama **DSP-FINAL-PROJECT** terdiri dari beberapa komponen penting berikut:

1. **main.py**, merupakan skrip utama yang mengelola alur aplikasi: dari pembukaan kamera, pemrosesan sinyal, hingga peluncuran GUI.
2. **gui.py**, membangun antarmuka pengguna berbasis **PyQt5**.
3. **signal_processing.py**, berisi logika pengolahan sinyal, seperti *filtering* dan transformasi Fourier (FFT) untuk estimasi HR dan RR.
4. **models/**, menyimpan model MediaPipe:
 - **blaze_face_short_range.tflite** untuk deteksi wajah.
 - **pose_landmarker.task** untuk deteksi *landmark* tubuh seperti bahu.
5. **icon/**, berisi aset grafis seperti **heart-icon.png** dan **lungs-icon.png** untuk tampilan GUI.
6. **README.md** dan **LICENSE**, berisi dokumentasi penggunaan dan lisensi proyek.
7. **requirements.txt**, mencantumkan *library* Python yang dibutuhkan, yaitu:
 - **opencv-python 4.8.1.78**
 - **mediapipe 0.10.7**
 - **PyQt5 5.15.9**

- numpy 1.24.3
- scipy 1.11.3
- matplotlib 3.7.2

Struktur ini memisahkan komponen GUI, pemrosesan sinyal, dan model pendeteksi secara modular, sehingga memudahkan proses pengembangan dan pemeliharaan sistem.

3.1 Instalasi Library

Agar program dapat berjalan dengan baik, library yang disebutkan pada penjelasan diatas harus di install. Karena library yang dibutuhkan sudah dibuat pada file "requirements.txt", kita dapat menginstall library dengan cara menuliskan perintah dibawah ini pada terminal : 29

```
1 pip install -r requirements.txt
```

Kode 1: Menginstal library melalui requirements.txt

3.2 Modul main.py

3.2.1 Import library pada main.py

Library yang sudah diinstal sebelumnya harus diimport terlebih dahulu agar dapat digunakan. Library yang digunakan adalah sys, numpy, openCV (cv2), mediapipe, os, scipy, PyQt5.QtWidgets (untuk QApplication dan QMainWindow), PyQt5.QtGui (untuk QImage dan QPixmap), dan PyQt5.QtCore (untuk Qt dan QTimer).

```
1 import sys
2 import cv2
3 import numpy as np
4 import mediapipe as mp
5 from PyQt5.QtWidgets import QApplication, QMainWindow
6 from PyQt5.QtGui import QImage, QPixmap
7 from PyQt5.QtCore import Qt, QTimer
8 import os
```

Kode 2: memanggil library yang telah diinstall

3.2.2 Memeriksa Dependensi

```
1 try:
2     from utils.gui import HealthTrackerUI
3     from utils.signal_processing import HealthAnalyzer # Kelas utama untuk pemrosesan
4     sinyal
5 except ImportError as e:
6     print(f"Penting: Gagal mengimpor modul dari folder 'utils'. Pastikan file ada dan benar
7     : {e}")
8     print("Harap buat file 'utils/gui.py' dan 'utils/signal_processing.py' sesuai kebutuhan
9     .")
10    sys.exit(1)
```

Kode 3: Memeriksa dependensi

kode diatas berfungsi untuk memeriksa apakah dependensi HealthTrackerUI dari modul gui.py dan dependensi HealthAnalyzer dari modul signal_processing tersedia atau tidak. Jika tidak tersedia, akan memunculkan pesan eror, seperti menyarankan pengguna untuk memastikan file-file tersebut ada dan benar atau menyarankan untuk membuat file-file yang hilang. Blok try sendiri digunakan dalam Python untuk menangani potensi kesalahan yang mungkin terjadi selama eksekusi kode di dalamnya.

3.2.3 Fungsi draw_landmarks_on_image

Fungsi `draw_landmarks_on_image` digunakan untuk menampilkan visualisasi landmark tubuh yang terdeteksi oleh MediaPipe pada gambar RGB. Fungsi ini sangat berguna dalam proses debugging karena memungkinkan pengembang melihat secara langsung posisi titik-titik pose, dalam proyek ini digunakan untuk bahu yang dikenali oleh sistem. Di dalam fungsi ini, digunakan modul `drawing_utils` dari MediaPipe untuk menggambar koneksi antar titik pose menggunakan warna dan spesifikasi garis tertentu.

```

1     def draw_landmarks_on_image(rgb_image, detection_result):
2         """Menggambar landmark pose pada gambar untuk debugging."""
3         from mediapipe.python.solutions import drawing_utils as mp_drawing
4         pose_landmarks_list = detection_result.pose_landmarks
5         annotated_image = np.copy(rgb_image)
6         for idx in range(len(pose_landmarks_list)):
7             pose_landmarks = pose_landmarks_list[idx]
8             mp_drawing.draw_landmarks(
9                 annotated_image,
10                pose_landmarks,
11                mp.solutions.pose.POSE_CONNECTIONS if hasattr(mp.solutions, 'pose') else None,
12                mp_drawing.DrawingSpec(color=(0,255,0), thickness=2, circle_radius=2),
13                mp_drawing.DrawingSpec(color=(0,0,255), thickness=2, circle_radius=2)
14            )
15         return annotated_image
16

```

Kode 4: Fungsi `draw_landmarks_on_image`

3.2.4 Kelas `MainWindow.__init__`

Fungsi ini bertugas membangun tampilan antarmuka pengguna (*HealthTrackerUI*), mengatur jalannya kamera, serta mempersiapkan alat analisis (*HealthAnalyzer*) untuk proses estimasi *Heart Rate* (HR) dan *Respiration Rate* (RR). Dalam fungsi ini, semua komponen penting seperti path model MediaPipe, *frame rate kamera* (FPS), tampilan plot Matplotlib, dan buffer sinyal rPPG dan pernapasan diatur.

29

```

1     class MainWindow(QMainWindow):
2         def __init__(self):
3             super().__init__()
4             self.setWindowTitle("Realtime rPPG & Respiration Rate Tracker by BEE Team")
5             self.ui = HealthTrackerUI()
6             self.setCentralWidget(self.ui)
7             self.setMinimumSize(1000, 600)
8
9             # Konfigurasi model & FPS
10            self.face_model_path_config = "models/blaze_face_short_range.tflite"
11            self.pose_model_path_config = "models/pose_landmarker.task"
12            self.fps_config = 30
13
14            # Inisialisasi HealthAnalyzer
15            self.analyzer = HealthAnalyzer(
16                face_model_path=self.face_model_path_config,
17                pose_model_path=self.pose_model_path_config,
18                fps=self.fps_config
19            )
20
21            # Cek model berhasil dimuat
22            if not self.analyzer.has_models():
23                raise RuntimeError("Model MediaPipe gagal dimuat di HealthAnalyzer.")
24

```

```

25     # Hubungkan komponen GUI
26     self.video_label = self.ui.video_label
27     self.hr_label = self.ui.hr_value_label
28     self.rr_label = self.ui.rr_value_label
29     self.ax_rppg = self.ui.ax_rppg
30     self.canvas_rppg = self.ui.hr_canvas
31     self.ax_resp = self.ui.ax_resp
32     self.canvas_resp = self.ui.rr_canvas
33
34     self.rppg_line, = self.ax_rppg.plot([], [], color='#FF6B6B')
35     self.resp_line, = self.ax_resp.plot([], [], color='#6BCBFF')
36
37     self.cap = None
38     self.timer = QTimer(self)
39     self.timer.timeout.connect(self.update_frame)
40
41     self.inference_interval = 3
42     self.frame_count_for_inference = 0
43     self.last_face_detection_result = None
44     self.last_pose_detection_result = None
45
46     self.process_interval = self.fps_config // 2
47     self.frames_since_last_process = 0
48     self.last_processed_hr = 0.0
49     self.last_processed_rr = 0.0
50     self.last_filtered_rppg = []
51     self.last_filtered_resp = []
52
53     self.ui.start_button.clicked.connect(self.start_processing)
54     self.ui.end_button.clicked.connect(self.end_processing)
55     self.ui.end_button.setEnabled(False)
56     self.video_label.setText("Tekan START untuk memulai feed kamera")
57

```

Kode 5: Fungsi MainWindow.__init__

3.2.5 Fungsi start_processing

Fungsi start_processing dijalankan ketika pengguna menekan tombol START di aplikasi. Fungsi ini bertugas untuk mengaktifkan webcam, memulai timer pembaruan frame sesuai FPS yang ditentukan, dan mereset seluruh buffer serta variabel status yang menyimpan sinyal dan hasil sebelumnya. Fungsi ini penting karena memastikan bahwa *buffer* sinyal rPPG dan RR telah dibersihkan sebelum mulai mengumpulkan data baru.

re29

```

1     def start_processing(self):
2         if self.analyzer is None or not self.analyzer.has_models():
3             self.ui.video_label.setText("Analyzer/Model tidak termuat. Proses tidak dapat dimulai.")
4         )
5         return
6
7         if self.cap is None:
8             self.cap = cv2.VideoCapture(0)
9         if not self.cap.isOpened():
10            self.ui.video_label.setText("Error: Tidak dapat membuka webcam!")
11            self.cap = None
12            return
13
14            self.timer.start(int(1000.0 / self.fps_config))
15            self.ui.start_button.setEnabled(False)
16            self.ui.end_button.setEnabled(True)

```

```

16         self.analyzer.clear_buffers()
17         self.frames_since_last_process = 0
18         self.last_processed_hr = 0.0
19         self.last_processed_rr = 0.0
20         self.last_filtered_rppg = []
21         self.last_filtered_resp = []
22
23         self.frame_count_for_inference = 0
24         self.last_face_detection_result = None
25         self.last_pose_detection_result = None
26
27         blank_frame = np.zeros((480, 640, 3), dtype=np.uint8)
28         self._update_gui_plots_and_labels(blank_frame, [], [], 0.0, 0.0, force_plot_update=True)
29
30
31

```

Kode 6: Fungsi start_processing

3.2.6 Fungsi end_processing

Fungsi `end_processing` dijalankan ketika pengguna menekan tombol END. Tugas utamanya adalah menghentikan aliran video dan proses analisis sinyal secara bersih. Fungsi ini akan menghentikan timer yang memicu pembaruan frame, menyalakan kamera, dan mereset tampilan GUI ke kondisi awal. Ini termasuk mengosongkan grafik plot sinyal rPPG dan RR, mengatur ulang nilai label detak jantung dan pernapasan, serta menampilkan pesan bahwa proses telah berakhir.

```

1     def end_processing(self):
2         self.timer.stop()
3         if self.cap is not None:
4             self.cap.release()
5             self.cap = None
6
7         self.ui.start_button.setEnabled(True)
8         self.ui.end_button.setEnabled(False)
9         self.video_label.setText("Feed Kamera Berakhir. Tekan START.")
10        self.hr_label.setText("-- BPM")
11        self.rr_label.setText("-- Breaths/min")
12
13        self.last_filtered_rppg, self.last_filtered_resp = [], []
14        self.last_processed_hr, self.last_processed_rr = 0.0, 0.0
15        self.rppg_line.set_data([], [])
16        self.resp_line.set_data([], [])
17
18        self.canvas_rppg.draw_idle()
19        self.canvas_resp.draw_idle()
20

```

Kode 7: Fungsi end_processing

3.2.7 Fungsi _preprocess_frame

Fungsi `_preprocess_frame` berfungsi sebagai langkah awal dalam alur pemrosesan video, yaitu mengambil satu frame dari webcam dan mempersiapkannya untuk dianalisis oleh MediaPipe. Pertama-tama, fungsi memeriksa apakah kamera aktif dan tersedia. Jika berhasil, frame dibaca dan kemudian dilakukan flip horizontal agar tampilannya menyerupai cermin, seperti kamera depan pada perangkat mobile. Selanjutnya, warna frame dikonversi dari format BGR ke RGB karena MediaPipe memerlukan format RGB untuk melakukan deteksi wajah dan pose. Frame RGB ini kemudian dibungkus ke dalam objek `mp.Image`, yang merupakan format input standar untuk pipeline MediaPipe. Fungsi

ini mengembalikan tiga nilai, yaitu frame asli (BGR), frame RGB, dan objek `mp.Image`. Ketiganya digunakan untuk tahapan selanjutnya.

```

1      def _preprocess_frame(self):
2          if self.cap is None or not self.cap.isOpened():
3              return None, None, None
4
5          ret, frame = self.cap.read()
6          if not ret:
7              return None, None, None
8
9          frame = cv2.flip(frame, 1)
10         rgb_frame_for_mp = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
11         mp_image = mp.Image(image_format=mp.ImageFormat.SRGB, data=rgb_frame_for_mp)
12         return frame, rgb_frame_for_mp, mp_image
13
14

```

Kode 8: Fungsi `_preprocess_frame`

3.2.8 Fungsi `_update_gui_plots_and_labels`

Fungsi `_update_gui_plots_and_labels` menyinkronkan hasil pemrosesan data dengan tampilan antarmuka. Fungsi ini memperbarui grafik sinyal rPPG dan pernapasan, serta label HR dan RR secara real-time. Selain itu, fungsi ini mengonversi frame OpenCV ke QImage agar video bisa ditampilkan di QLabel PyQt5. Fungsi ini menjadi penghubung utama antara data dan tampilan GUI.

```

1      def _update_gui_plots_and_labels(self, frame_processed, filtered_rppg, filtered_resp,
2          hr, rr, force_plot_update=False):
3          if filtered_rppg or force_plot_update:
4              self.rppg_line.set_ydata(filtered_rppg)
5              self.rppg_line.set_xdata(range(len(filtered_rppg)))
6              self.ax_rppg.relim(); self.ax_rppg.autoscale_view(True, True, True)
7              self.canvas_rppg.draw_idle()
8
9          if filtered_resp or force_plot_update:
10             self.resp_line.set_ydata(filtered_resp)
11             self.resp_line.set_xdata(range(len(filtered_resp)))
12             self.ax_resp.relim(); self.ax_resp.autoscale_view(True, True, True)
13             self.canvas_resp.draw_idle()
14
15             self.hr_label.setText(f"{hr:.0f} BPM" if hr > 0 else "-- BPM")
16             self.rr_label.setText(f"{rr:.0f} Breaths/min" if rr > 0 else "-- Breaths/min")
17
18             if frame_processed is not None and frame_processed.size > 0:
19                 display_frame_for_qt = cv2.cvtColor(frame_processed, cv2.COLOR_BGR2RGB)
20                 h, w, ch = display_frame_for_qt.shape
21                 bytes_per_line = ch * w
22                 qt_image = QImage(display_frame_for_qt.data, w, h, bytes_per_line, QImage.Format_RGB888)
23             )
24             pixmap = QPixmap.fromImage(qt_image)
25             self.video_label.setPixmap(pixmap.scaled(self.video_label.size(), Qt.KeepAspectRatio,

```

Kode 9: Fungsi `_update_gui_plots_and_labels`

3.2.9 Fungsi `update_frame`


```

1  def update_frame(self):
2      if self.analyzer is None:
3          return
4
5      original_frame_bgr, _, mp_image = self._preprocess_frame()
6      if original_frame_bgr is None:
7          self._update_gui_plots_and_labels(None, self.last_filtered_rppg, self.
last_filtered_resp, self.last_processed_hr, self.last_processed_rr)
8          return
9
10     frame_to_display_with_roi = original_frame_bgr.copy()
11
12     run_inference_this_frame = (self.frame_count_for_inference %
self.inference_interval == 0)
13     self.frame_count_for_inference += 1
14
15     if run_inference_this_frame and mp_image:
16         self.last_face_detection_result = self.analyzer.detect_faces(mp_image)
17         self.last_pose_detection_result = self.analyzer.detect_pose(mp_image)
18
19         if self.last_face_detection_result:
20             self.analyzer.process_rppg_from_face(frame_to_display_with_roi, self.
last_face_detection_result)
21         if self.last_pose_detection_result:
22             self.analyzer.process_respiration_from_pose(frame_to_display_with_roi, self.
last_pose_detection_result)
23
24         self.frames_since_last_process += 1
25         if self.frames_since_last_process >= self.process_interval:
26             self.frames_since_last_process = 0
27             if len(self.analyzer.rppg_signal_buffer) >= self.analyzer.min_signal_length:
28                 self.last_filtered_rppg, self.last_processed_hr = self.analyzer.
filter_and_calculate_hr()
29             if len(self.analyzer.resp_signal_buffer) >= self.analyzer.min_signal_length:
30                 self.last_filtered_resp, self.last_processed_rr = self.analyzer.
filter_and_calculate_rr()
31
32         self._update_gui_plots_and_labels(frame_to_display_with_roi, self.last_filtered_rppg, self.
last_filtered_resp, self.last_processed_hr, self.last_processed_rr)
33
34

```

Kode 10: Fungsi update_frame

3.2.10 Fungsi closeEvent

Fungsi closeEvent dipanggil otomatis saat jendela aplikasi ditutup, dan berfungsi untuk menghentikan pemrosesan video, timer, serta melepaskan resource seperti kamera. Fungsi ini memanggil `end_processing`, mencetak log penutupan, dan menerima event penutupan dengan `event.accept()`, guna mencegah konflik saat aplikasi dijalankan ulang.

```

1  def closeEvent(self, event):
2      self.end_processing()
3      print("Aplikasi ditutup.")
4      event.accept()
5
6

```

Kode 11: Fungsi closeEvent label

3.3 Modul `signal_processing.py`

3.3.1 *Import Library*

```

1 import cv2
2 import numpy as np
3 import mediapipe as mp
4 from mediapipe.tasks import python as mp_python
5 from mediapipe.tasks.python import vision as mp_vision
6 import scipy.signal as signal
7 import os

```

Kode 12: Import Library

3.3.2 Fungsi `extract_rppg_signal`

Fungsi `extract_rppg_signal` mengekstraksi sinyal rPPG dengan menghitung rata-rata nilai channel hijau dari ROI pada frame video, karena channel hijau paling sensitif terhadap aliran darah. Fungsi ini menerima frame dan koordinat ROI, lalu mengembalikan nilai rata-rata hijau atau None jika ROI tidak valid. Fungsi ini penting untuk mengubah perubahan warna wajah menjadi data numerik.

```

1 def extract_rppg_signal(frame, roi):
2     """
3     Mengekstraksi nilai rata-rata channel hijau dari Region of Interest (ROI) pada sebuah frame
4     .
5     Args:
6         frame (numpy.ndarray): Frame video input dalam format BGR.
7         roi (tuple): Tuple berisi (x, y, w, h) yang mendefinisikan ROI.
8     Returns:
9         float or None: Nilai rata-rata channel hijau di ROI, atau None jika ROI tidak valid.
10    """
11    x, y, w, h = roi
12    if w > 0 and h > 0:
13        roi_frame = frame[y:y+h, x:x+w]
14        mean_green = cv2.mean(roi_frame)[1] # Channel hijau (indeks 1 di BGR)
15        return mean_green
16    return None

```

Kode 13: Fungsi `extract_rppg_signal`

3.3.3 Fungsi `butter_bandpass`

Fungsi `butter_bandpass` digunakan untuk membuat filter Butterworth bandpass dengan frekuensi *cut-off* dan sampling tertentu. Filter ini menghasilkan koefisien (b, a) untuk menyaring noise dan hanya mempertahankan frekuensi relevan, seperti untuk detak jantung dan pernapasan. Tanpa filter ini, estimasi HR dan RR bisa terganggu oleh gangguan eksternal.

```

1 def butter_bandpass(lowcut, highcut, fs, order=5):
2     """
3     Mendesain koefisien filter bandpass Butterworth.
4     Args:
5         lowcut (float): Frekuensi cut-off bawah (Hz).
6         highcut (float): Frekuensi cut-off atas (Hz).
7         fs (float): Frekuensi sampling sinyal (Hz).
8         order (int): Orde filter.
9     Returns:
10        tuple: Koefisien numerator (b) dan denominator (a) dari filter.
11    """
12    nyq = 0.5 * fs # Frekuensi Nyquist

```

```

13     low = lowcut / nyq
14     high = highcut / nyq
15     b, a = signal.butter(order, [low, high], btype='band')
16     return b, a
17

```

Kode 14: Fungsi band_pass

3.3.4 Fungsi calculate_rate_from_fft

Fungsi *calculate_rate_from_fft* ini bertujuan untuk menghitung laju detak jantung atau pernapasan dari sinyal dengan cara menganalisis frekuensinya menggunakan *Fast Fourier Transform (FFT)*. Pertama tama, fungsi akan memeriksa apakah sinyal (*signal_value*) memiliki panjang yang cukup untuk analisis FFT, jika tidak mencukupi maka fungsi akan mengembalikan nilai 0. Jika data memiliki panjang sinyal yang mencukupi, maka fungsi akan menghitung panjang sinyal pada nilai N dan akan mengubahnya menjadi bentuk frekuensi (*yf xf*) menggunakan *np.fft.fft*. Dari frekuensi yang didapat, fungsi akan mengambil frekuensi di antara *low cut* dan *high cut*.

```

1  def calculate_rate_from_fft(signal_values, fs, lowcut_hz, highcut_hz):
2      """
3      Menghitung laju (detak jantung/pernapasan) dari sinyal menggunakan FFT.
4      Args:
5          signal_values (list/numpy.ndarray): Nilai sinyal.
6          fs (float): Frekuensi sampling (Hz).
7          lowcut_hz (float): Batas bawah rentang frekuensi yang diminati (Hz).
8          highcut_hz (float): Batas atas rentang frekuensi yang diminati (Hz).
9      Returns:
10         float: Laju yang dihitung dalam satuan per menit (mis., BPM).
11     """
12     if len(signal_values) < 20: # Membutuhkan panjang sinyal yang cukup untuk analisis FFT
13         return 0
14
15     N = len(signal_values)
16     yf = np.fft.fft(signal_values)
17     xf = np.fft.fftfreq(N, 1 / fs)
18
19     # Ambil hanya spektrum positif dan normalisasi amplitudo
20     xf_positive = xf[:N//2]
21     yf_positive = 2.0/N * np.abs(yf[0:N//2])
22
23     # Cari indeks frekuensi dalam rentang yang valid
24     valid_indices = np.where((xf_positive >= lowcut_hz) & (xf_positive <= highcut_hz))[0]
25
26     if len(valid_indices) == 0: # Tidak ada frekuensi dalam rentang valid
27         return 0
28
29     valid_yf = yf_positive[valid_indices]
30     valid_xf = xf_positive[valid_indices]
31
32     if len(valid_yf) == 0: # Pengecekan tambahan
33         return 0
34
35     # Temukan frekuensi dengan amplitudo terbesar (puncak dominan)
36     dominant_peak_index_in_valid = np.argmax(valid_yf)
37     dominant_frequency_hz = valid_xf[dominant_peak_index_in_valid]
38
39     rate_per_minute = dominant_frequency_hz * 60 # Konversi Hz ke per menit
40     return rate_per_minute
41

```

Kode 15: Fungsi calculate_rate_from_fft

3.3.5 Fungsi `__init__` dalam kelas `HealthAnalyzer`

Fungsi `__init__` yang ada pada kelas `HealthAnalyzer` berfungsi untuk melakukan konfigurasi awal. Fungsi ini menyimpan parameter penting seperti frame per second (fps) dari video input, serta menghitung dan menyimpan panjang minimum sinyal (`min_signal_length`). Pada fungsi ini juga telah ditetapkan untuk batas frekuensi bawah (`lowcut`) dan atas (`highcut`) untuk pemrosesan sinyal remote photoplethysmography (rPPG) dan sinyal pernapasan. Selain itu, fungsi ini melakukan proses pemuatan model machine learning dari MediaPipe untuk deteksi wajah (`face_detector`) dan deteksi pose (`pos_landmarker`).

```

1  class HealthAnalyzer:
2      """
3      Kelas untuk menganalisis sinyal biologis dari input video, termasuk deteksi wajah,
4      pose, ekstraksi sinyal rPPG dan pernapasan, serta perhitungan laju terkait.
5      """
6      def __init__(self, face_model_path="models/blaze_face_short_range.tflite",
7                  pose_model_path="models/pose_landmarker.task",
8                  fps=30,
9                  rppg_lowcut=0.75, rppg_highcut=4.0,
10                 resp_lowcut=0.1, resp_highcut=0.7,
11                 min_signal_length_factor=2,
12                 frame_buffer_factor=10):
13         """
14         Inisialisasi HealthAnalyzer.
15         Args:
16             face_model_path (str): Path ke model deteksi wajah MediaPipe.
17             pose_model_path (str): Path ke model deteksi pose MediaPipe.
18             fps (int): Frame per second dari input video.
19             rppg_lowcut (float), rppg_highcut (float): Batas frekuensi untuk sinyal rPPG (Hz).
20             resp_lowcut (float), resp_highcut (float): Batas frekuensi untuk sinyal pernapasan
21             (Hz).
22             min_signal_length_factor (int): Pengali FPS untuk menentukan panjang sinyal minimum
23             .
24             frame_buffer_factor (int): Pengali FPS untuk menentukan batas buffer sinyal.
25         """
26         self.fps = fps
27         self.min_signal_length = int(min_signal_length_factor * self.fps)
28         self.frame_buffer_limit = int(frame_buffer_factor * self.fps)
29
30         self.rppg_lowcut = rppg_lowcut
31         self.rppg_highcut = rppg_highcut
32         self.resp_lowcut = resp_lowcut
33         self.resp_highcut = resp_highcut
34
35         self.face_detector = None
36         self.pose_landmarker = None
37         self._load_models(face_model_path, pose_model_path) # Muat model MediaPipe
38
39         # Desain koefisien filter untuk rPPG dan pernapasan
40         self.rppg_b, self.rppg_a = butter_bandpass(self.rppg_lowcut, self.rppg_highcut, self.
41         fps)
42         self.resp_b, self.resp_a = butter_bandpass(self.resp_lowcut, self.resp_highcut, self.
43         fps)
44
45         # Buffer untuk menyimpan sinyal mentah yang diekstrak
46         self.rppg_signal_buffer = []
47         self.resp_signal_buffer = []

```

Kode 16: Fungsi `__init__`

3.3.6 Fungsi `_load_models`

Fungsi `_load_models` ini berfungsi untuk memuat dua model, yaitu model untuk mendeteksi wajah (FaceDetector) dan model untuk mendeteksi landmark atau titik-titik pada pose tubuh (PoseLandmarker). Fungsi ini pertama-tama memastikan file model deteksi wajah (`face_model_path`) tersedia, kemudian mengatur opsi spesifik untuk FaceDetector seperti mode pemrosesan per gambar (RunningMode.IMAGE). FaceDetector kemudian dibuat dan disimpan dalam `self.face_detector`. Pada model untuk mendeteksi landmark, fungsi ini mengatur konfigurasi untuk PoseLandmarker, yang mencakup mode pemrosesan per gambar dan jumlah maksimal pose yang akan dideteksi. PoseLandmarker lalu dibuat dari konfigurasi tersebut dan disimpan dalam `self.pose_landmarker`.

```

1     def _load_models(self, face_model_path, pose_model_path):
2         """
3         Memuat model deteksi wajah dan pose MediaPipe dari path yang diberikan.
4         """
5         try:
6             # Inisialisasi FaceDetector
7             if not os.path.exists(face_model_path):
8                 raise FileNotFoundError(f"File model wajah tidak ditemukan: {face_model_path}")
9             face_base_options = mp_python.BaseOptions(model_asset_path=face_model_path)
10            face_options = mp_vision.FaceDetectorOptions(
11                base_options=face_base_options,
12                running_mode=mp_vision.RunningMode.IMAGE,
13                min_detection_confidence=0.5
14            )
15            self.face_detector = mp_vision.FaceDetector.create_from_options(face_options)
16
17            # Inisialisasi PoseLandmarker
18            if not os.path.exists(pose_model_path):
19                raise FileNotFoundError(f"File model pose tidak ditemukan: {pose_model_path}")
20            pose_base_options = mp_python.BaseOptions(model_asset_path=pose_model_path)
21            pose_options = mp_vision.PoseLandmarkerOptions(
22                base_options=pose_base_options,
23                running_mode=mp_vision.RunningMode.IMAGE,
24                num_poses=1,
25                min_pose_detection_confidence=0.5,
26                min_tracking_confidence=0.5
27            )
28            self.pose_landmarker = mp_vision.PoseLandmarker.create_from_options(pose_options)
29            print("Model MediaPipe di HealthAnalyzer berhasil dimuat.")
30        except Exception as e:
31            print(f"Error saat memuat model MediaPipe di HealthAnalyzer: {e}")
32            raise e
33

```

Kode 17: Fungsi `_load_models`

3.3.7 Fungsi `detect_faces`

Secara sederhana, fungsi `detect_faces` ini berfungsi untuk menggunakan model deteksi wajah yang sudah dimuat sebelumnya pada (`self.face_detector`) untuk menemukan wajah dalam sebuah gambar yang diberikan dalam format MediaPipe (`mp_image`). Fungsi `detect_faces` berada pada kelas `HealthAnalyzer` dan akan dipanggil ketika perlu melakukan deteksi wajah.

```

1     def detect_faces(self, mp_image):
2         """
3         Melakukan deteksi wajah pada gambar MediaPipe.
4         Args:
5             mp_image (mp.Image): Gambar dalam format MediaPipe.
6         Returns:

```

```

7         Hasil deteksi wajah dari MediaPipe atau None jika gagal.
8         """
9         if self.face_detector:
10             try:
11                 return self.face_detector.detect(mp_image)
12             except Exception as e:
13                 print(f"Error deteksi wajah: {e}")
14         return None
15

```

Kode 18: Fungsi detec_faces

3.3.8 Fungsi detect_pose

Fungsi detect_pose ini tidak jauh berbeda dengan fungsi detec_faces, namun menggunakan model deteksi pose (self.pose_landmarker) yang sudah dimuat sebelumnya untuk menemukan dan mengidentifikasi landmark pada pose tubuh manusia dalam sebuah gambar yang diberikan dalam format MediaPipe .

```

1     def detect_pose(self, mp_image):
2         """
3         Melakukan deteksi pose pada gambar MediaPipe.
4         Args:
5             mp_image (mp.Image): Gambar dalam format MediaPipe.
6         Returns:
7             Hasil deteksi pose dari MediaPipe atau None jika gagal.
8         """
9         if self.pose_landmarker:
10             try:
11                 return self.pose_landmarker.detect(mp_image)
12             except Exception as e:
13                 print(f"Error deteksi pose: {e}")
14         return None
15

```

Kode 19: Fungsi detect_pose

3.3.9 Fungsi process_rppg_from_face

Fungsi ini mengekstraksi sinyal rPPG dari wajah pengguna, khususnya dari ROI dahi. Fungsi ini pertama-tama menentukan bounding box wajah dari hasil deteksi, kemudian memotong ROI yang mewakili dahi, menghitung rata-rata warna hijau, lalu menambahkannya ke *buffer* sinyal rPPG.

```

1     def process_rppg_from_face(self, frame_for_signal, face_detection_result):
2         """
3         Mengekstrak sinyal rPPG dari ROI wajah (dahi),
4         menggambar ROI pada frame, dan menambahkan rata-rata sinyal ke buffer.
5         Args:
6             frame_for_signal (numpy.ndarray): Frame video (BGR) untuk ekstraksi dan
            penggambaran.
7             face_detection_result: Hasil deteksi wajah dari MediaPipe.
8         Returns:
9             float or None: Nilai rPPG mentah yang baru diekstrak, atau None jika gagal.
10        """
11        if face_detection_result is None or not face_detection_result.detections:
12            return None
13
14        # Dapatkan bounding box utama wajah
15        detection = face_detection_result.detections[0]
16        bbox = detection.bounding_box
17        frame_h, frame_w, _ = frame_for_signal.shape

```

```

18     x, y, w, h = int(bbox.origin_x), int(bbox.origin_y), int(bbox.width), int(bbox.height)
19
20     # Validasi bounding box utama
21     x = max(0, min(x, frame_w - 1))
22     y = max(0, min(y, frame_h - 1))
23     w = max(0, min(w, frame_w - x))
24     h = max(0, min(h, frame_h - y))
25
26     if not (w > 0 and h > 0):
27         return None
28
29     extracted_signals = []
30     # Definisi, validasi, ekstraksi, dan penggambaran untuk ROI Dahi
31     fh_x, fh_y, fh_w, fh_h = int(x+w*0.25), int(y+h*(-0.1)), int(w*0.5), int(h*0.20)
32     fh_x, fh_y = max(0, min(fh_x, frame_w-1)), max(0, min(fh_y, frame_h-1))
33     fh_w, fh_h = max(0, min(fh_w, frame_w-fh_x)), max(0, min(fh_h, frame_h-fh_y))
34     if fh_w > 0 and fh_h > 0:
35         val = extract_rppg_signal(frame_for_signal, (fh_x, fh_y, fh_w, fh_h))
36         if val is not None: extracted_signals.append(val)
37         cv2.rectangle(frame_for_signal, (fh_x, fh_y), (fh_x + fh_w, fh_y + fh_h), (0, 255,
255), 1) # Cyan
38
39     if extracted_signals:
40         rppg_value = np.mean(extracted_signals)
41         self.rppg_signal_buffer.append(rppg_value)
42         if len(self.rppg_signal_buffer) > self.frame_buffer_limit:
43             self.rppg_signal_buffer.pop(0)
44         return rppg_value
45     return None
46

```

Kode 20: Fungsi process_rppg_from_face

3.3.10 Fungsi process_respiration_from_pose

Fungsi ini mengekstraksi sinyal pernapasan berdasarkan pergerakan vertikal dari bahu kanan dan kiri. Menggunakan landmark ke-11 dan ke-12 dari hasil deteksi pose, fungsi ini menghitung rata-rata posisi y dari kedua bahu, lalu menambahkannya ke buffer sinyal pernapasan. Nilai ini dibalik (inversi) agar puncak naik mewakili inhalasi. Fungsi ini juga menggambar titik bahu pada frame video.

```

1     def process_respiration_from_pose(self,
2         frame_for_signal, pose_detection_result):
3         """
4         Mengekstrak sinyal pernapasan dari gerakan bahu, menggambar landmark pada frame,
5         dan menambahkan sinyal ke buffer.
6         Args:
7             frame_for_signal (numpy.ndarray): Frame video (BGR) untuk ekstraksi dan
penggambaran.
8             pose_detection_result: Hasil deteksi pose dari MediaPipe.
9         Returns:
10            float or None: Nilai sinyal pernapasan mentah yang baru diekstrak, atau None jika
gagal.
11        """
12        if pose_detection_result is None or not pose_detection_result.pose_landmarks:
13            return None
14
15        landmarks = pose_detection_result.pose_landmarks[0]
16        h_img, w_img, _ = frame_for_signal.shape
17        try:
18            # Pastikan landmark bahu (indeks 11 dan 12) terdeteksi dengan baik
19            if len(landmarks) > max(11,12) and \

```

```

20         hasattr(landmarks[11], 'visibility') and landmarks[11].visibility > 0.5 and \
21         hasattr(landmarks[12], 'visibility') and landmarks[12].visibility > 0.5:
22             rs_landmark = landmarks[12] # Bahu kanan
23             ls_landmark = landmarks[11] # Bahu kiri
24             y1_r, y1_l = int(rs_landmark.y * h_img), int(ls_landmark.y * h_img)
25
26             # Gambar lingkaran pada landmark bahu untuk visualisasi
27             cv2.circle(frame_for_signal, (int(rs_landmark.x*w_img), y1_r), 3, (255,0,0), -1) #
28             cv2.circle(frame_for_signal, (int(ls_landmark.x*w_img), y1_l), 3, (0,255,0), -1) #
29
30             avg_y_shoulder = np.mean([y1_r, y1_l])
31             self.resp_signal_buffer.append(-avg_y_shoulder) # Inversi agar puncak napas positif
32             if len(self.resp_signal_buffer) > self.frame_buffer_limit:
33                 self.resp_signal_buffer.pop(0)
34             return -avg_y_shoulder
35         except (IndexError, AttributeError) as e:
36             print(f"Peringatan: Gagal memproses landmark bahu di HealthAnalyzer: {e}")
37         return None
38

```

Kode 21: Fungsi process_respiration_from_pose

3.3.11 Fungsi filter_and_calculate_hr

Fungsi ini memproses buffer sinyal rPPG menggunakan filter /textitbandpass dan menghitung HR menggunakan FFT. Jika sinyal cukup panjang, maka dilakukan filtering menggunakan scipy.signal.filtfilt, lalu hasilnya dikirim ke fungsi FFT. Fungsi ini mengembalikan sinyal terfilter dan nilai HR dalam BPM.

```

1     def filter_and_calculate_hr(self):
2         """
3         Memfilter sinyal rPPG yang ada di buffer dan menghitung Heart Rate (HR).
4         Returns:
5         tuple: (list sinyal rPPG terfilter, float nilai HR dalam BPM).
6         """
7         if len(self.rppg_signal_buffer) < self.min_signal_length:
8             return list(self.rppg_signal_buffer), 0.0 # Kembalikan buffer mentah jika terlalu
9         pendek
10        try:
11            # Tentukan panjang padding untuk filtfilt, hindari error jika sinyal terlalu pendek
12            padlen = min(self.min_signal_length - 1, len(self.rppg_signal_buffer) - 1)
13            if padlen <= 0: # Tidak cukup data untuk filtfilt yang stabil
14                filtered_signal = list(self.rppg_signal_buffer)
15            else:
16                filtered_signal = signal.filtfilt(self.rppg_b, self.rppg_a, self.
17                rppg_signal_buffer, padlen=padlen).tolist()
18            hr = calculate_rate_from_fft(filtered_signal, self.fps, self.rppg_lowcut, self.
19            rppg_highcut)
20            return filtered_signal, hr
21        except ValueError: # Jika terjadi error saat filtering/FFT
22            return list(self.rppg_signal_buffer), 0.0

```

Kode 22: Fungsi filter_and_calculate_hr

3.3.12 Fungsi filter_and_calculate_rr

Fungsi ini bekerja sama seperti filter_and_calculate_hr, tetapi untuk buffer sinyal pernapasan. Sinyal difilter dengan bandpass 0.1–0.7 Hz, lalu diubah menjadi nilai RR (*breaths per minute*) menggunakan

FFT. Fungsi ini sangat penting untuk mendapatkan estimasi pernapasan yang halus dan akurat.

```

1  def filter_and_calculate_rr(self):
2      """
3      Memfilter sinyal pernapasan yang ada di buffer dan menghitung Respiration Rate (RR).
4      Returns:
5      tuple: (list sinyal pernapasan terfilter, float nilai RR dalam Breaths/min).
6      """
7      if len(self.resp_signal_buffer) < self.min_signal_length:
8          return list(self.resp_signal_buffer), 0.0 # Kembalikan buffer mentah jika terlalu
pendek
9      try:
10         padlen = min(self.min_signal_length -1, len(self.resp_signal_buffer)-1)
11         if padlen <= 0: # Tidak cukup data untuk filtfilt yang stabil
12             filtered_signal = list(self.resp_signal_buffer)
13         else:
14             filtered_signal = signal.filtfilt(self.resp_b, self.resp_a, self.resp_signal_buffer
, padlen=padlen).tolist()
15
16         rr = calculate_rate_from_fft(filtered_signal, self.fps, self.resp_lowcut, self.
resp_highcut)
17         return filtered_signal, rr
18     except ValueError: # Jika terjadi error saat filtering/FFT
19         return list(self.resp_signal_buffer), 0.0
20

```

Kode 23: Fungsi filter_and_calculate_rr

3.3.13 Fungsi clear_buffers

Fungsi clear_buffers ini cukup sederhana, yang dimana berfungsi untuk mengosongkan atau membersihkan semua data yang tersimpan dalam *buffer* sinyal rPPG dan buffer sinyal pernapasan.

```

1  def clear_buffers(self):
2      """
3      Menghapus semua data dari buffer sinyal rPPG dan pernapasan.
4      """
5      self.rppg_signal_buffer.clear()
6      self.resp_signal_buffer.clear()
7

```

Kode 24: Fungsi clear_buffers

3.4 Fungsi has_models

Fungsi has_models ini berfungsi untuk memastikan kedua model MediaPipe, yaitu untuk deteksi wajah dan deteksi pose, sudah berhasil dimuat dan siap digunakan

29

```

1  def has_models(self):
2      """
3      Memeriksa apakah model MediaPipe telah berhasil dimuat.
4      Returns:
5      bool: True jika kedua model (wajah dan pose) telah dimuat, False jika tidak.
6      """
7      return self.face_detector is not None and self.pose_landmarker is not None
8

```

Kode 25: Fungsi has_models

3.5 Modul gui.py

3.5.1 Import Library

- **PyQt5.QtWidgets**: Komponen utama untuk membangun tampilan (label, tombol, layout).
- **QPixmap**: Untuk memuat ikon seperti gambar hati dan paru-paru.
- **Qt**: Untuk mengatur alignment, scaling, dan flags lain.
- **matplotlib**: Untuk menampilkan grafik sinyal secara real-time (HR dan RR).

```

1  import os
2  from PyQt5.QtWidgets import (QWidget, QLabel, QPushButton, QVBoxLayout, QHBoxLayout,
3                               QGridLayout, QGroupBox, QApplication, QSizePolicy)
4  from PyQt5.QtGui import QFont, QPixmap
5  from PyQt5.QtCore import Qt
6  from matplotlib.backends.backend_qt5agg import FigureCanvasQTAgg as FigureCanvas
7  import matplotlib.pyplot as plt
8

```

Kode 26: Import Library

3.5.2 Kelas HealthTrackerUI

Merupakan widget utama yang mendefinisikan seluruh elemen tampilan dari aplikasi. Diturunkan dari QWidget, kelas ini menjadi pusat dari layout dan visualisasi GUI.

```

1  class HealthTrackerUI(QWidget):
2      """
3      Kelas untuk mendefinisikan dan mengatur elemen-elemen Antarmuka Grafis Pengguna (GUI)
4      untu
5

```

Kode 27: Kelas HealthTrackerUI

3.5.3 Fungsi _init_ui

Membuat layout utama vertikal (main_layout) sebagai struktur dasar aplikasi.

```

1  def _init_ui(self):
2      self.main_layout = QVBoxLayout(self)
3

```

Kode 28: Fungsi _init_ui

3.5.4 Fungsi _apply_styles

- Warna latar belakang gelap untuk keseluruhan aplikasi.
- Warna **hijau** untuk tombol **START**, dan **oranye** untuk tombol **END**.
- Label nilai HR ditampilkan dengan warna **merah muda**, dan RR dengan warna **biru muda**.
- Plot **Matplotlib** disesuaikan warnanya agar kontras terhadap latar belakang gelap.

```

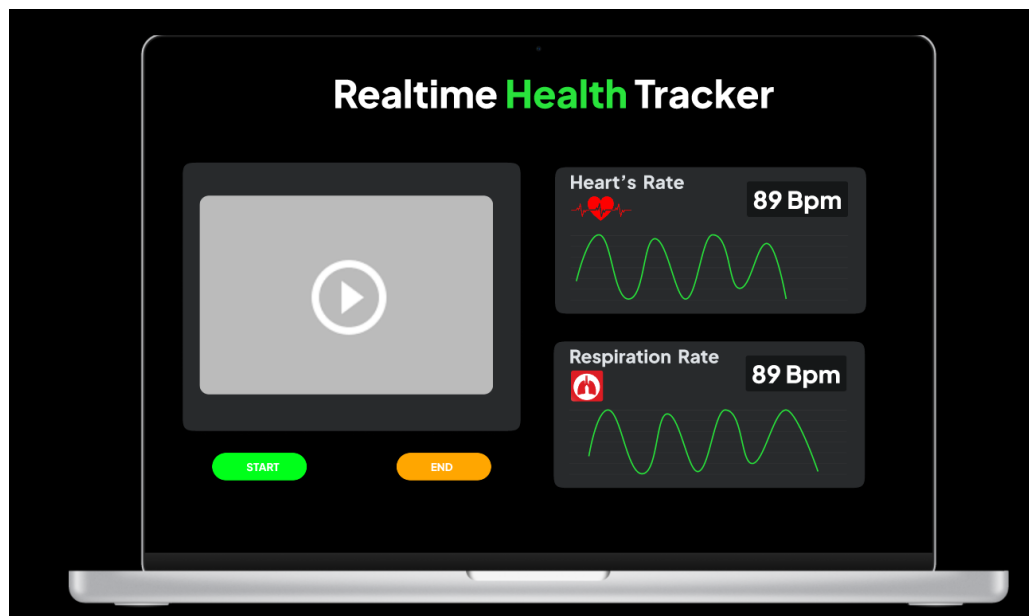
1  def _apply_styles(self):
2      self.setStyleSheet("""
3          QWidget {
4              background-color: #1E1E1E;
5              color: #E0E0E0;
6          }
7          QLabel#TitleLabel {
8              font-size: 28px;
9              font-weight: bold;
10             color: #66BB6A;
11         }
12         QPushButton#StartButton {
13             background-color: #4CAF50;
14         }
15         QPushButton#EndButton {
16             background-color: #FF9800;
17         }
18     """)
19

```

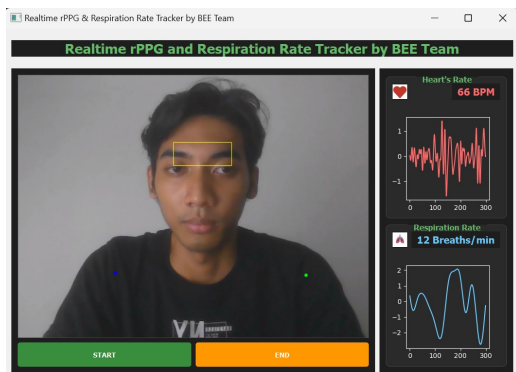
Kode 29: Fungsi `_apply_styles`

3.5.5 Komponen Non Fungsi

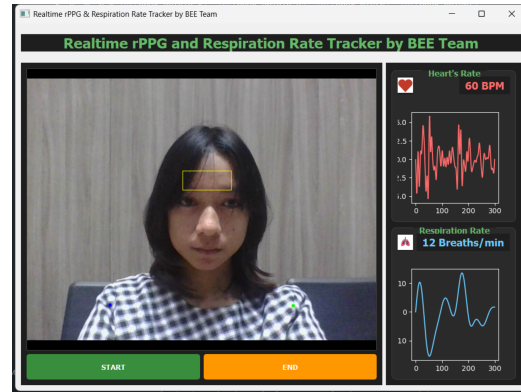
Antarmuka (UI) pengguna aplikasi ini terdiri dari tampilan video real-time dari webcam di sebelah kiri, disertai tombol START dan END untuk mengendalikan proses. Di sebelah kanan terdapat bagian yang menampilkan informasi detak jantung dan laju pernapasan, masing-masing dilengkapi dengan ikon visual (jantung dan paru-paru), label nilai dalam satuan BPM dan Breaths/min, serta grafik sinyal yang diperbarui secara real-time menggunakan Matplotlib. Grafik detak jantung menunjukkan sinyal rPPG dari area wajah, sedangkan grafik pernapasan merepresentasikan pergerakan bahu. Desain antarmuka ini dibuat terlebih dahulu menggunakan [Figma](#) sebelum diimplementasikan ke dalam aplikasi.



Gambar 2: Desain UI menggunakan Figma



(a) Hasil Percobaan 1



(b) Hasil Percobaan 2



(c) Hasil Percobaan 3

Gambar 3: Hasil Estimasi HR dan RR

4 Hasil

Area dahi dipilih sebagai lokasi utama untuk ekstraksi sinyal *Heart Rate* (HR) karena merupakan bagian wajah yang umumnya memiliki pencahayaan yang merata, minim gerakan otot, dan relatif bebas dari elemen pengganggu seperti ekspresi wajah atau rambut. Selain itu, area ini memiliki konsistensi warna kulit yang baik untuk menangkap fluktuasi mikroskopis akibat aliran darah, sehingga cocok untuk metode pengolahan sinyal berbasis rPPG seperti POS (*Plane-to-Orthogonal Skin*).

Untuk estimasi *Respiration Rate* (RR), dua titik referensi ditempatkan pada bahu kiri dan bahu kanan. Pemilihan ini didasarkan pada fakta bahwa gerakan bahu mencerminkan ekspansi dan kontraksi rongga dada saat bernapas. Dengan melacak perubahan posisi vertikal dari kedua landmark bahu menggunakan model pose dari MediaPipe, sistem dapat mendeteksi siklus pernapasan secara real-time dan tanpa kontak langsung.

Berdasarkan hasil pengamatan dari sistem yang telah diimplementasikan, nilai *Heart Rate* (HR) yang diperoleh rata-rata berada pada kisaran 60 hingga 72 BPM (*Beats Per Minute*). Rentang ini sesuai dengan nilai detak jantung normal bagi orang dewasa dalam kondisi istirahat, yang menurut Jensen et al. berada dalam batas fisiologis yang sehat dan stabil. Hal ini menunjukkan bahwa sistem mampu melakukan ekstraksi sinyal rPPG secara akurat dari video webcam menggunakan metode *Plane-to-Orthogonal Skin* (POS).

Sementara itu, nilai *Respiration Rate* (RR) yang terpantau rata-rata sebesar **12 breaths per minute**, juga sesuai dengan standar laju pernapasan normal pada orang dewasa sehat yaitu sekitar 12–20 napas per menit. Hasil ini mengindikasikan bahwa metode ekstraksi sinyal pernapasan berbasis landmark bahu yang diterapkan melalui deteksi pose dengan MediaPipe berjalan efektif. Estimasi HR dan RR yang akurat ini sejalan dengan pendekatan-pendekatan sebelumnya dalam studi pengukuran *vital signs*

berbasis kamera tanpa kontak langsung [1, 2].

5 Kesimpulan

Berdasarkan hasil implementasi dan pengujian sistem estimasi *Heart Rate (HR)* dan *Respiration Rate (RR)* menggunakan metode *Plane-to-Orthogonal Skin (POS)* dengan bantuan MediaPipe dan webcam, dapat disimpulkan bahwa sistem mampu melakukan ekstraksi sinyal fisiologis secara non-invasif dan real-time. Rata-rata nilai HR yang terdeteksi berada dalam rentang 60 hingga 72 BPM, sedangkan RR berada di sekitar 12 breaths per minute. Kedua nilai tersebut berada dalam kisaran normal untuk orang dewasa dalam kondisi istirahat, menunjukkan bahwa sistem memiliki tingkat akurasi dan kestabilan yang baik. Penggunaan ROI pada dahi untuk rPPG dan pergerakan bahu untuk RR terbukti efektif dalam mengestimasi parameter vital tanpa memerlukan perangkat khusus tambahan. Oleh karena itu, sistem ini berpotensi untuk dikembangkan lebih lanjut sebagai solusi monitoring kesehatan berbasis kamera yang praktis, murah, dan dapat digunakan secara luas dalam konteks pelayanan kesehatan digital maupun pemantauan mandiri.

References

- [1] B. Zhang, H. Li, L. Xu, L. Qi, Y. Yao, and S. E. Greenwald, "Noncontact heart rate measurement using a webcam, based on joint blind source separation and a skin reflection model: For a wide range of imaging conditions," *Journal of Sensors*, vol. 2021, pp. 1–18, 2021.
- [2] J. Lee, H. Joo, and J. Woo, "Improved remote photoplethysmography using machine learning-based filter bank," *Applied Sciences*, vol. 14, no. 23, p. 11107, 2024.

6 Lampiran

Selama proses pengerjaan proyek tugas besar ini, anggota tim memanfaatkan bantuan teknologi AI sebagai alat pendukung dalam beberapa aspek. AI digunakan untuk membantu dalam penyusunan laporan serta menyusun struktur penulisan agar lebih sistematis dan mudah dipahami. Selain itu, AI juga dimanfaatkan dalam proses pengembangan kode program, seperti memberikan referensi sintaks, penjelasan fungsi-fungsi dalam pemrograman, serta saran perbaikan atau optimalisasi kode. Berikut adalah tautan dan *screenshot* yang mendokumentasikan penggunaan AI dalam proyek ini:

1. [Google Drive](#)
2. [Gemini 1](#)
3. [Gemini 2](#)
4. [ChatGPT](#)

Berikut dilampirkan dokumentasi Github: [Github](#)